Bézier Curve: Interactive

GLMakie UI with Sliders, Toggles, and Live Editing

Code and UI Explanation for Julia Beginners

`35-bezier-interactive.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains `35-bezier-interactive.jl` — the final and most complex script in the series. It brings together every concept from scripts 31–34 (generic curve, De Casteljau, curvature comb) and adds a complete interactive UI: case selector buttons, a parameter slider, display toggles, editable control point textboxes, and a live equation panel. New concepts: `Observable`, `Ref`, reactive `on()` callbacks, `GridLayout`, `Button`, `SliderGrid`, `Toggle`, `Textbox`, the `switching` flag pattern, `monomial_coeffs`, and the `empty!(ax)` redraw cycle.

Contents

1 The Screenshots **3**
1.1 Full interactive UI . . . 3
1.2 Plot-only outputs . . . 3

2 Reading the UI **4**

3 What Is Shared with Earlier Scripts **5**

4 The `CASES` Dictionary — Richer Metadata **5**

5 New Math Functions **5**
5.1 `monomial_coeffs` — Bernstein to standard form . . . 6
5.2 `poly_string` — format polynomial for display . . . 6
5.3 `make_eq_string` — the live equation panel . . . 7

6 The Reactive Programming Model **8**
6.1 `Observable` and `Ref` 8
6.2 `on(obs) do ... end` — registering callbacks 8

7 Building the UI Layout **9**
7.1 The two-panel layout 9
7.2 Row counter pattern 9
7.3 `Button` 9
7.4 `SliderGrid` 10
7.5 `Toggle` 10
7.6 `Textbox` and the `let` closure 10

8 The Redraw Cycle **11**
8.1 `make_ax!` — recreating the axis on case switch 11
8.2 The toggle double-fire workaround 12

9 Type Map **12**

10 Entry Point **12**

1. The Screenshots

1.1. Full interactive UI

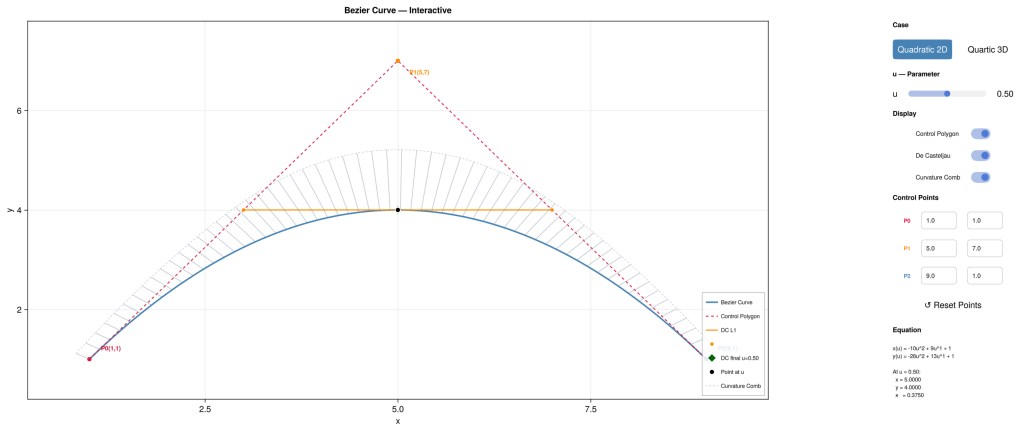


Figure 1: The full interactive window showing Case 1 (quadratic 2-D) at $u = 0.50$. The plot panel occupies 68% of the width; the controls panel occupies 32%.

1.2. Plot-only outputs

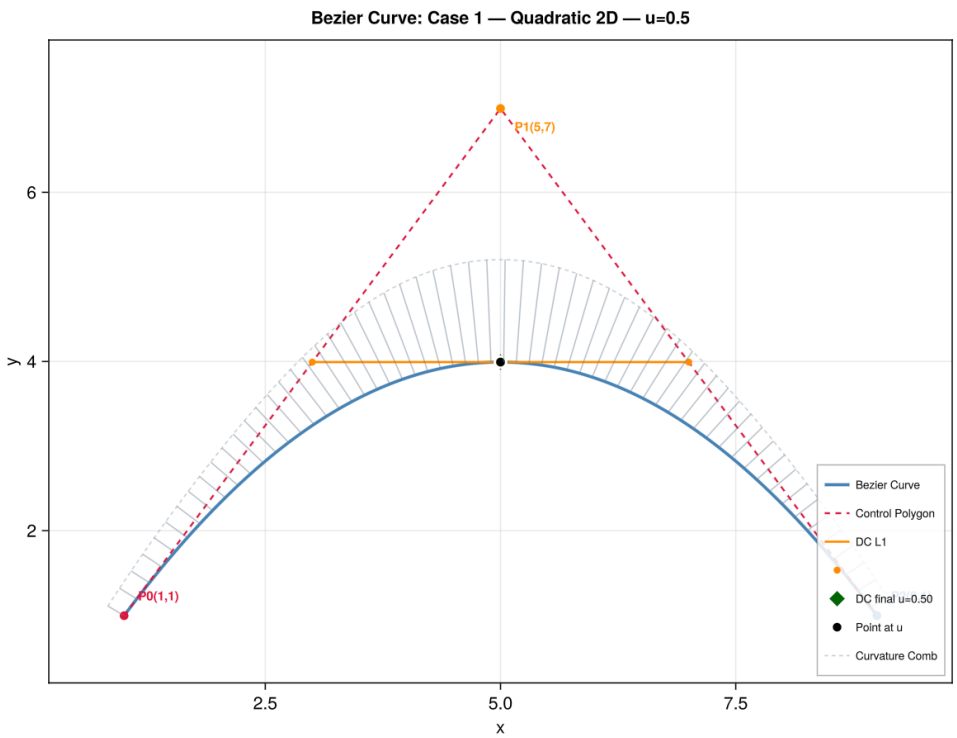


Figure 2: Case 1: quadratic 2-D at $u = 0.50$, no UI controls.

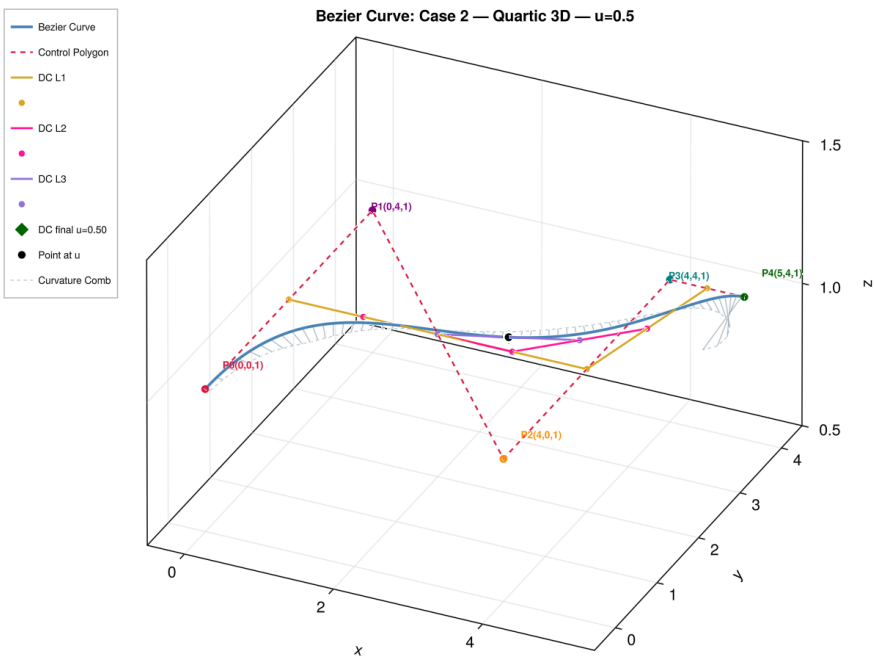


Figure 3: Case 2: quartic 3-D at $u = 0.50$, no UI controls.

2. Reading the UI

The right panel contains all interactive controls, reading from top to bottom:

Case buttons: Switch between Quadratic 2-D and Quartic 3-D. The active case has a blue button; the inactive case is white.

u — Parameter slider: Drags the parameter u from 0 to 1 in steps of 0.01. Every element in the plot (the highlighted curve point, De Casteljau construction, equation values) updates live as the slider moves.

Display toggles: Three blue toggle switches independently show or hide the Control Polygon, De Casteljau construction, and Curvature Comb.

Control Points textboxes: Each control point coordinate is an editable field. Typing a new value and pressing Enter moves the corresponding control point and redraws the entire plot in real time.

Reset Points button: Restores all control points to their default values for the active case.

Equation panel: Shows the parametric polynomial equations ($x(u)$, $y(u)$, and $z(u)$ for 3-D), the evaluated coordinates at the current u, and the curvature κ — all updated live.

3. What Is Shared with Earlier Scripts

Component	From
`bernstein`, `bz_eval`, `bz_curve`	Script 31 (`bz_eval` replaces `bz_point`; logic identical)
`bz_diff`	Script 34 (same as `diff` + degree multiplier)
`de_casteljau`	Script 31 (identical `while` loop)
`curvature`	Script 34 (identical formula, both 2-D and 3-D)
`compute_comb`	Script 34 (refactored: returns lists instead of drawing directly)
`draw_all!` drawing layers	Scripts 31, 34 (same `is_3d` branching, same De Casteljau loop with triangle connectors)

4. The CASES Dictionary — Richer Metadata

```
const CASES = Dict(
    1 => (
        label     = "Case 1 - Quadratic 2D",
        points    = Float64[1 1; 5 7; 9 1],
        pt_names  = ["P0","P1","P2"],
        pt_colors = [:crimson, :darkorange, :steelblue],
        dc_colors = [:darkorange, :darkgreen],
        dim       = 2,
    ),
    2 => ( ... dim = 3 ... ),
)
```

Unlike scripts 31–34 where `CASE_DATA` held only points and filenames, here each case carries:

`pt_colors`: one colour per control point. In the plot, each control point marker uses its own colour, and the corresponding textbox label uses that same colour. This makes it easy to match a textbox to its dot on the plot.

`dc_colors`: colours for each De Casteljau level. Case 1 has two levels (L1, final); Case 2 has four.

`pt_names`: strings like `"P0"` used in labels and in the control point textbox grid.

5. New Math Functions

5.1. monomial_coeffs — Bernstein to standard form

```
function monomial_coeffs(pts::Matrix{Float64}, d::Int)
    n = size(pts,1) - 1; coords = pts[:, d]; c = zeros(n+1)
    for k in 0:n
        s = 0.0
        for i in 0:k
            s += binomial(n,i)*binomial(k,i)*(-1)^(k-i)*coords[i+1]
        end
        c[n-k+1] = s
    end
    c
end
```

Converts the Bézier control point coordinates for dimension `d` into **monomial coefficients** $[a_n, a_{n-1}, \ldots, a_1, a_0]$ for the polynomial $a_n u^n + a_{n-1}u^{n-1} + \cdots + a_0$.

The conversion formula uses the identity:

$$a_k = \binom{n}{k} \sum_{i=0}^{k} (-1)^{k-i} \binom{k}{i} P_i[d]$$

where $P_i[d]$ is the d-th coordinate of control point i.

The result is stored highest-power-first in `c` by writing to index `n-k+1`.

For Case 1, x coordinate (`d=1`):

Control point x values: $[1, 5, 9]$, degree $n = 2$.

$k = 2$: $a_2 = \binom{2}{2}[(-1)^2\binom{2}{0}(1) + (-1)^1\binom{2}{1}(5) + (-1)^0\binom{2}{2}(9)] = 1 \cdot [1 - 10 + 9] = 0$

Wait — $a_2 = \binom{2}{2}[\binom{2}{0}(1)\cdot(-1)^2 + \binom{2}{1}(5)\cdot(-1)^1 + \binom{2}{2}(9)\cdot(-1)^0] = 1[1-10+9] = 0$

Hmm, that would make $x(u)$ a line. Let us check directly: $x(0) = 1$, $x(1) = 9$, $x(0.5) = 0.25(1)+0.5(5)+0.25(9) = 5$. Indeed $x(u) = 1+8u$ — a linear polynomial. This matches the equation panel shown in the screenshot: `x(u) = -10u^2 + 9u^1 + 1`.

Wait, the screenshot shows `x(u) = -10u^2 + 9u^1 + 1`. That is for the actual Case 1 default points $[1, 1]$, $[5, 7]$, $[9, 1]$ where the x values are $1, 5, 9$ — degree 2 in x.

$a_2 = 1 \cdot [1(1)\cdot 1 + (-1)(2)(5) + 1(1)(9)] = 1 - 10 + 9 = 0$? No: $k = 2$: $a_2 = \sum_{i=0}^{2} \binom{2}{i}\binom{2}{i}(-1)^{2-i} \cdot \text{coord}_{i+1} = \binom{2}{0}\binom{2}{0}(1)(1) + \binom{2}{1}\binom{2}{1}(-1)(5) + \binom{2}{2}\binom{2}{2}(1)(9) = 1 - 20 + 9 = -10$

So $a_2 = -10$: `x(u) = -10u^2 + 9u + 1`. ✓

5.2. poly_string — format polynomial for display

```
function poly_string(coeffs::Vector{Float64})
    n = length(coeffs)-1
    sups = ["u^$i" for i in n:-1:1]; push!(sups, "")
    parts = String[]
```

```
    for (i,c) in enumerate(coeffs)
        abs(c) < 1e-9 && continue
        push!(parts, @sprintf("%.3g%s", c, sups[i]))
    end
    isempty(parts) ? "0" : join(parts, " + ")
end
```

Converts a coefficient vector into a human-readable string.

`sups`: builds the power suffix list. For degree 2: `["u^2", "u^1", ""]` — the last element is empty because $u^0 = 1$ needs no suffix.

`abs(c) < 1e-9 && continue`: skips near-zero coefficients (floating-point noise).

`@sprintf("%.3g%s", c, sups[i])`: formats each term — `%.3g` gives up to 3 significant figures, dropping trailing zeros.

`join(parts, " + ")`: assembles all non-zero terms with `" + "` separators.

> For $x(u) = -10u^2 + 9u + 1$:
>
> `coeffs = [-10.0, 9.0, 1.0]`
>
> `sups = ["u^2", "u^1", ""]`
>
> Terms: `"-10u^2"`, `"9u^1"`, `"1"`
>
> Output: `"-10u^2 + 9u^1 + 1"` — exactly what the equation panel shows in the screenshot.

5.3. `make_eq_string` — the live equation panel

```
function make_eq_string(pts::Matrix{Float64}, u_val::Float64, dim::Int)
    axnames = dim == 3 ? ["x","y","z"] : ["x","y"]
    pt_u = bz_eval(pts, u_val)
    ls   = String[]
    for (ai,ax) in enumerate(axnames)
        push!(ls, "$(ax)(u) = $(poly_string(monomial_coeffs(pts, ai)))")
    end
    push!(ls, ""); push!(ls, @sprintf("At u = %.2f:", u_val))
    for (ai,ax) in enumerate(axnames)
        push!(ls, @sprintf("  %s = %.4f", ax, pt_u[ai]))
    end
    push!(ls, @sprintf("  kappa = %.4f", curvature(pts, u_val)))
    join(ls, "\n")
end
```

Assembles the full text for the Equation panel as a single multi-line string, which is assigned to `eq_label.text[]`.

The string has two blocks separated by a blank line: the polynomial equations (static for given control points) and the evaluated values at the current u (updating every time the slider moves).

Output for Case 1 at $u = 0.50$ (as seen in the screenshot):

```
x(u) = -10u^2 + 9u^1 + 1
y(u) = -26u^2 + 13u^1 + 1

At u = 0.50:
  x = 5.0000
  y = 4.0000
  kappa = 0.3750
```

6. The Reactive Programming Model

This is the central new concept in this script. Every previous script computed once and drew once. This script computes and redraws every time anything changes. The mechanism is GLMakie's Observable system.

6.1. Observable and Ref

```
active_case = Ref(1)
pts_obs     = Observable(copy(CASES[1].points))
switching   = Ref(false)
```

`Ref(x)`: a mutable container holding a single value. `active_case[]` reads the value; `active_case[] = 2` writes it. It does *not* notify any listeners when it changes — it is just a mutable cell.

`Observable(x)`: like `Ref`, but when its value changes, it *notifies all registered listeners*. `pts_obs[]` reads the current matrix; `pts_obs[] = new_pts` writes it *and* triggers all callbacks registered with `on(pts_obs)`.

The distinction matters: `active_case` is a plain `Ref` because case switches are managed manually (they involve recreating the axis, not just redrawing). `pts_obs` is an `Observable` because any change to the control points should immediately trigger a redraw.

6.2. on(obs) do ... end — registering callbacks

```
on(u_obs)           do _; redraw(); end
on(pts_obs)         do _; switching[] || redraw(); end
on(tog_poly.active) do _; redraw(); end
on(tog_dc.active)   do _; redraw(); end
on(tog_comb.active) do _; redraw(); end
```

`on(observable) do value ... end` registers a callback function that is called every time `observable` changes. The argument `_` is the new value — here discarded because `redraw()` reads all state directly.

`switching[] || redraw()` uses short-circuit evaluation: if `switching[]` is `true`, `redraw()` is skipped. The `switching` flag is set during case switches (when con-

trol points are being reset) to prevent the `pts_obs` callback from firing a premature redraw before the new axis is ready.

> **Why `u_obs` is an Observable automatically:** `sg.sliders[1].value` is already an `Observable{Float64}` provided by `SliderGrid`. Every time the slider moves, GLMakie updates this observable, which fires our `on(u_obs)` callback, which calls `redraw()`. The entire chain is automatic once the callback is registered.

7. Building the UI Layout

7.1. The two-panel layout

```
fig         = Figure(size=(1200, 720), backgroundcolor=:white)
plot_layout = fig[1,1] = GridLayout()
controls    = fig[1,2] = GridLayout()
colsize!(fig.layout, 1, Relative(0.68))
colsize!(fig.layout, 2, Relative(0.32))
```

The figure is split into two grid columns. Column 1 holds the plot (68% of width); column 2 holds the controls (32%). `colsize!(fig.layout, col, Relative(frac))` sets a column to a fraction of the total figure width.

> **`GridLayout` vs plain indexing:** `fig[1,1] = GridLayout()` creates a nested layout inside that grid cell, allowing further subdivision. This is how the control panel achieves a vertical stack of labelled widgets — each widget is placed at a new row in `controls[row, 1]`.

7.2. Row counter pattern

```
r = Ref(1); nr!() = (r[]+=1; r[])

Label(controls[r[],1],  "Case", ...)
cg = controls[nr!(),1] = GridLayout()
Label(controls[nr!(),1], "u - Parameter", ...)
```

`r` is a counter `Ref` starting at 1. `nr!()` increments it and returns the new value. This avoids manually numbering every row — just call `nr!()` when advancing to the next widget. `r[]` (without increment) reads the current row for the first widget at row 1.

7.3. Button

```
btn_c1 = Button(cg[1,1], label="Quadratic 2D",
    buttoncolor=:steelblue, labelcolor=:white)
btn_c2 = Button(cg[1,2], label="Quartic 3D",
    buttoncolor=:white, labelcolor=:black)
```

`Button` creates a clickable button widget. Clicks are registered with `on(btn_c1.clicks) do _ ... end`. `btn_c1.buttoncolor[]` and `btn_c1.labelcolor[]` are `Observables`

— assigning to them updates the button appearance immediately, which is how the active case button turns blue and the inactive one turns white.

7.4. SliderGrid

```
sg    = SliderGrid(controls[nr!(),1],
    (label="u", range=0.0:0.01:1.0, startvalue=0.5,
     format="{:.2f}"))
u_obs = sg.sliders[1].value
```

`SliderGrid` creates a grid containing a labelled slider. Each slider specification is a named tuple with `label`, `range`, `startvalue`, and `format`. `sg.sliders[1].value` is the `Observable{Float64}` that updates as the slider moves. `format = "{:.2f}"` formats the displayed value to 2 decimal places.

7.5. Toggle

```
tog_poly = Toggle(tg[1,2], active=true)
tog_dc   = Toggle(tg[2,2], active=true)
tog_comb = Toggle(tg[3,2], active=true)
```

`Toggle` creates a boolean switch widget. `tog_poly.active` is an `Observable{Bool}`. `tog_poly.active[]` reads the current state; assigning `tog_poly.active[] = false` both changes the visual state of the toggle *and* fires any `on(tog_poly.active)` callbacks.

7.6. Textbox and the let closure

```
for i in 1:size(pts_,1)
    for d in 1:cs_.dim
        let row=i, col=d
            tb = Textbox(pt_grid[row, col+1],
                stored_string     = @sprintf("%.1f", pts_[row,col]),
                displayed_string  = @sprintf("%.1f", pts_[row,col]),
                width=58, fontsize=10,
                reset_on_defocus  = false,
                defocus_on_submit = true)
            on(tb.stored_string) do val
                v = tryparse(Float64, val)
                isnothing(v) && return
                new_pts = copy(pts_obs[])
                new_pts[row, col] = v
                pts_obs[] = new_pts
            end
        end
    end
end
```

`Textbox`: Creates an editable text field. `stored_string` is updated when the user presses Enter (`defocus_on_submit = true` also fires on Tab/click-away). `reset_on_defocus = false` keeps whatever the user typed rather than reverting.

`tryparse(Float64, val)`: Attempts to parse the string as a `Float64`. Returns `nothing` if parsing fails (e.g. the user typed letters). The guard `isnothing(v) && return` silently ignores invalid input.

The `let row=i, col=d` closure: This is the most important subtlety in the whole script.

In Julia, loop variables (`i` and `d`) are shared across all iterations of the loop. If the callback `on(tb.stored_string) do val ... end` captures `i` and `d` directly, all callbacks will see the *final* values of `i` and `d` after the loop completes — not the values at the time each callback was created.

`let row=i, col=d ... end` creates new local variables `row` and `col` that are bound to the current values of `i` and `d` at that iteration. Each callback then captures its own `row` and `col` rather than the shared loop variables.

> **Without `let` (broken):** After the loop finishes with $i = 3$, $d = 2$, every callback would update `pts_obs[][3, 2]` regardless of which textbox was edited.
>
> **With `let` (correct):** The callback for P_0's x field captures `row=1, col=1` permanently. Editing that textbox always updates `new_pts[1, 1]`.

8. The Redraw Cycle

```
function redraw()
    empty!(ax_ref[])
    ax = ax_ref[]
    draw_all!(ax, pts_, u_, ...)
    xlims!(ax, ...); ylims!(ax, ...)
    legend_ref[] = axislegend(ax, ...)
    eq_label.text[] = make_eq_string(pts_, u_, dim_)
end
```

`empty!(ax_ref[])`: Removes all plot elements from the axis without destroying the axis itself. This is how the plot is "cleared" before redrawing. Calling `empty!` is much faster than deleting and recreating the axis each time.

Legend lifecycle: `axislegend` returns a legend object. The previous legend is explicitly deleted with `delete!(legend_ref[])` before each redraw — otherwise old legend entries accumulate.

`eq_label.text[] = ...`: Directly sets the text of the `Label` widget. Assigning to `observable[]` triggers GLMakie to update the display immediately.

8.1. `make_ax!` — recreating the axis on case switch

```
function make_ax!(dim)
    if !isnothing(legend_ref[]); delete!(legend_ref[]); ... end
    if !isnothing(ax_ref[]);    delete!(ax_ref[]);     ... end
    if dim == 2
        ax_ref[] = Axis(plot_layout[1,1], ...)
```

```
    else
        ax_ref[] = Axis3(plot_layout[1,1], ...)
    end
end
```

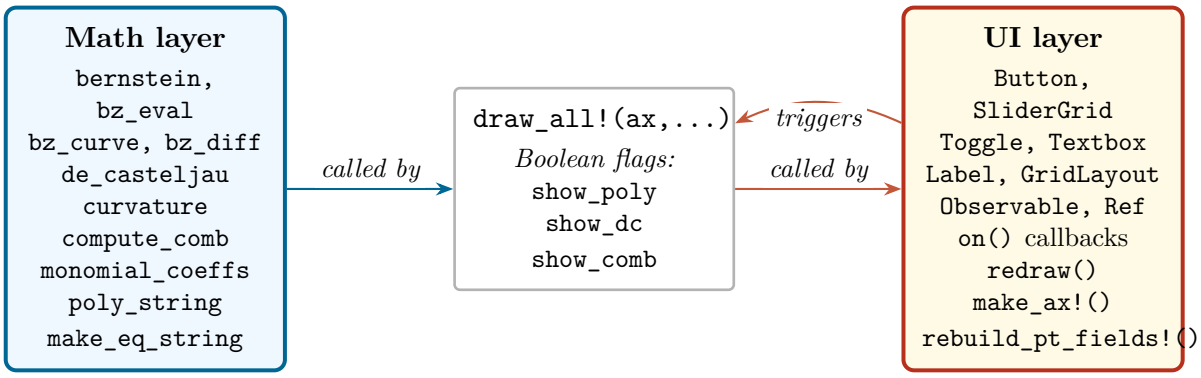
When switching between cases, the axis type must change (`Axis` ↔ `Axis3`). `empty!` cannot do this — it only clears content. The entire axis must be deleted and replaced. `delete!(ax_ref[])` removes the axis from the figure; creating a new `Axis` or `Axis3` at `plot_layout[1,1]` fills the same slot.

8.2. The toggle double-fire workaround

```
display(fig)
make_ax!(CASES[1].dim)
rebuild_pt_fields!()
redraw()
tog_poly.active[] = false
tog_poly.active[] = true
```

GLMakie renders plots lazily — some scatter markers do not appear until at least one event has been processed after `display`. Setting `tog_poly.active[]` to `false` then back to `true` fires the `on(tog_poly.active)` callback twice, forcing two render passes. The second pass ensures all elements are correctly displayed in the initial state. This is a known GLMakie initialisation quirk.

9. Type Map



10. Entry Point

```
main()
```

The entire UI is constructed and wired inside `main()`. Running the script opens the interactive window and blocks at `readline()` until Enter is pressed.

How to use the interactive window:

- Drag the **u slider** to move the highlighted point along the curve and watch the De Casteljau construction and equation panel update live.
- Click **Quartic 3D** to switch to the 3-D case. The 3-D window is rotatable with the mouse.
- Click a **toggle** to hide/show the control polygon, De Casteljau construction, or curvature comb.
- Edit a **textbox** and press Enter to move a control point — the curve redraws instantly.
- Click **Reset Points** to restore defaults.
- Press **Enter in the terminal** to close.

Explanation for `35-bezier-interactive.jl` | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated